



Today's agenda

↳ Grid illumination

↳ Brick wall



AlgoPrep

leetcode 1001 (hard)



Q) Grid illumination

↳ Given $N \times N$ matrix, $lamps[i] = \{x_i, y_i\}$ indicates that grid $\{x_i, y_i\}$ is turned on. On lamp illuminates the cell like queen. you will be given queries and answer accordingly.

	0	1	2	3	4
0	Q				
1					
2					
3					
4				Q	

$lamps = \{(0,0) (4,3)\}$

queries: $\{(1,1) (0,0)\}$
 ↓ ↓
 true false

Idea 1

row

	0	1	2	3	4
0	Q				
1					
2					
3					
4				Q	

$lamps = \{(0,0) (4,3)\}$

m1

row	count
0 - 1	
4 - 1	

m2

col	count
0 - 1	
3 - 1	

m3 (row+col)

0 - 1	
1 - 1	

m4 (row+col)

0 - 1	
7 - 1	

m5

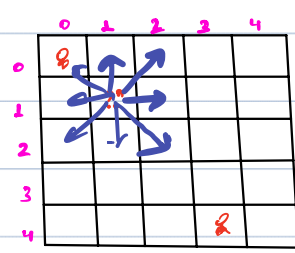
0 - 1	
28 - 1	



queries = $\{(1,1), (0,0)\}$

ans:

1	
---	--



mx = 1
my = 1

m1

pos = (0,0)

0 - 10
4 - 1

m2

pos = (0,0)

0 - 10
3 - 1

m3 (0,0)

0 - 10
1 - 1

m4 (0,0)

0 - 10
7 - 1

m5

25 - 1

int dirs[] = {
0 1
-1, 0
0, 1
1, 0
1, -1
0, -1
-1, -1
8 {0,0}}

d = 0 x1 = 0
 y1 = 1

d = 1 x1 = 0
 y1 = 1 + 1 = 2

d = 2 x1 = 1 + 0 = 1
 y1 = 1 + 1 = 2

d = 3 x1 = 1 + 1 = 2
 y1 = 1 + 1 = 2

d = 7 x1 = 1 - 1 = 0
 y1 = 1 - 1 = 0
 times = 1

```
for (d = 0; d < dirs.length; d++) {  
    int nx = x + dirs[d][0];  
    int ny = y + dirs[d][1];  
    if (nx >= 0 && ny >= 0 && nx < N && ny < N &&  
        ms.containsKey(nx * N + ny)) {  
        int times = ms.get(nx * N + ny);  
        m1.put (nx, m1.getOrDefault(nx, 0) - times);  
        m2.put (ny, m2.getOrDefault(ny, 0) - times);  
        m3.put (nx - y, m3.getOrDefault(nx - y, 0) - times);  
        m4.put (nx * y, m4.getOrDefault(nx * y, 0) - times);  
        m5.remove (nx * N + ny);  
    }  
}
```



//Pseudo code

```
int[] gridillumination (int n, int[][] lamps, int[][] queries) {
    HashMap < Integer, Integer > m1, m2, m3, m4, m5;

    for (int i=0; i < lamps.length; i++) {
        int x = lamps[i][0];
        int y = lamps[i][1];
        m1.put (x, m1.getOrDefault (x, 0) + 1);
        m2.put (y, m2.getOrDefault (y, 0) + 1);
        m3.put (x-y, m3.getOrDefault (x-y, 0) + 1);
        m4.put (x+y, m4.getOrDefault (x+y, 0) + 1);
        m5.put (x*n+y, m5.getOrDefault (x*n+y, 0) + 1);
    }

    int[] ans = new length[queries.length];
```

```
int [][] dir = { {0,-1,0}, {-1,-1,1}, {0,1,1}, {1,1,1}, {1,0,1}, {1,-1,1},
```

```
               {0,-1,1}, {-1,-1,0}, {0,0,1} }
```

time: $O(L+Q)$

s.c: $O(L)$

```
for (int i=0; i < queries.length; i++) {
```

```
    int x = queries[i][0];
```

```
    int y = queries[i][1];
```

```
    if (m1.getOrDefault (x, 0) > 0 || m2.getOrDefault (y, 0) > 0 ||
        m3.getOrDefault (x-y, 0) > 0 || m4.getOrDefault (x+y, 0) > 0
```

```
        ans[i] = 1;
```

```
}
```



```
for (d=0; d < dirs.length; d++) {
```

```
    int x1 = x + dirs[d][0];
```

```
    int y1 = y + dirs[d][1];
```

```
    if (x1 >= 0 && y1 >= 0 && x1 < N && y1 < N &&
```

```
        m5.containsKey(x1 * N + y1) == true) {
```

```
        int times = m5.get(x1 * N + y1);
```

```
        m1.put(x1, m1.getOrDefault(x1, 0) - times);
```

```
        m2.put(y1, m2.getOrDefault(y1, 0) - times);
```

```
        m3.put(x1 * y1, m3.getOrDefault(x1 * y1, 0) - times);
```

```
        m4.put(x1 * y1, m4.getOrDefault(x1 * y1, 0) - times);
```

```
        m5.remove(x1 * N + y1);
```

```
    }
```

```
}
```

```
}
```

```
return ans;
```

```
}
```



```

for (int i=0; i< lamps.length; i++) {
    int x = lamps[i][0];
    int y = lamps[i][1];
    m1.put(x, m1.getOrDefault(x,0)+1);
    m2.put(y, m2.getOrDefault(y,0)+1);
    m3.put(x-y, m3.getOrDefault(x-y,0)+1);
    m4.put(x+y, m4.getOrDefault(x+y,0)+1);
}

```

	0	1
0	1	1
1	1	2
2	3	1
3	5	3
4	7	1

$6 \rightarrow 1$
 $7 \rightarrow 1$
 $16 \rightarrow 1$

$x = 1$
 $y = 2$

m1 (column)

1 - 1	2 - 1
-------	-------

m2 (column)

1 - 1	2 - 1
-------	-------

m3 (row-col)

0 - 1	-1 - 1
-------	--------

m4 (row+col)

2 - 1	3 - 1
-------	-------

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19
4	20	21	22	23	24

$\rightarrow (i, j) \rightarrow i * 5 + j$

$(3, 2) \rightarrow 3 * 5 + 2 = 17$

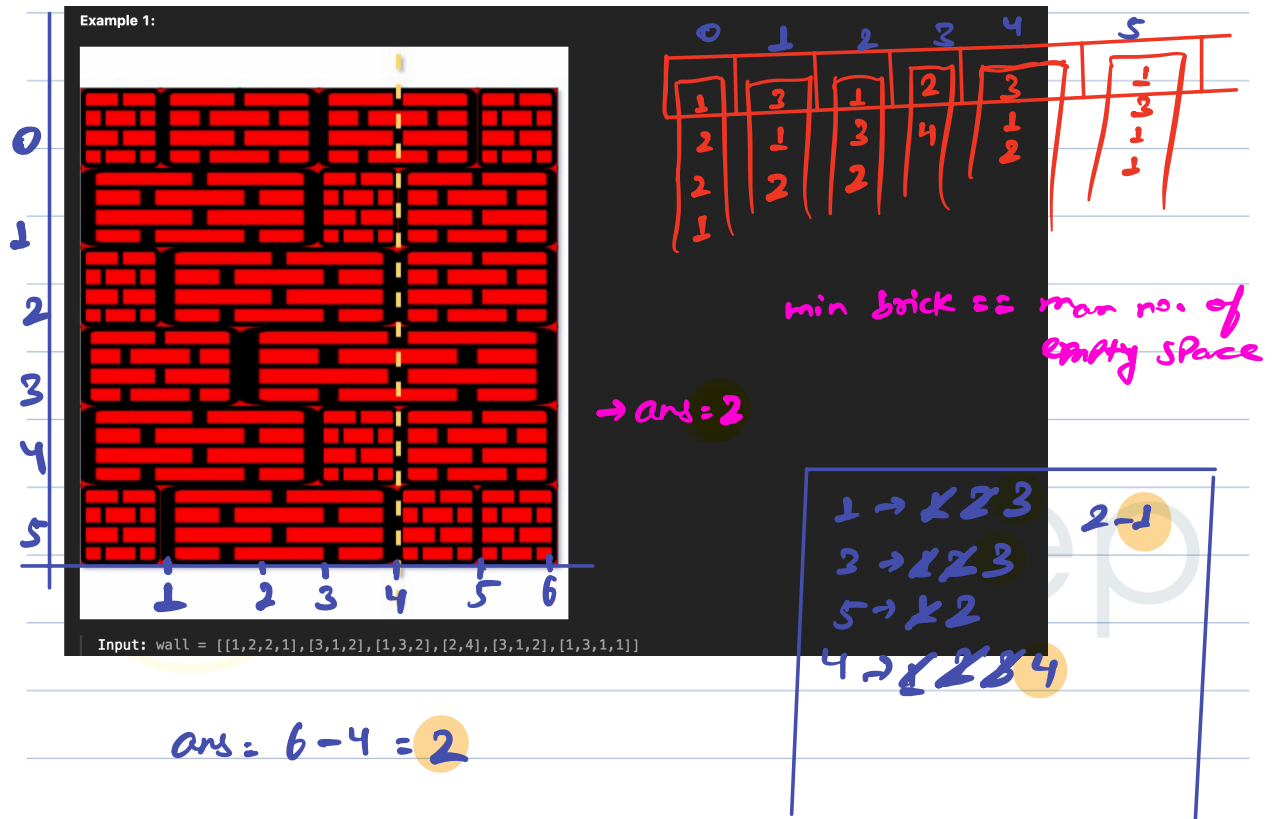
$(1, 3) \rightarrow 1 * 5 + 3 = 8$

leetcode 554 (med)



Q) Brick wall

↳ Given a rectangular brick, Draw a vertical line crossing least bricks.





//Pseudo code

```
int leastbricks (List<List<Integer>> walls){  
    HashMap<Integer, Integer> map = new HashMap<>();  
    int ans = 0;  
    for (List<Integer> biglist : walls) {  
        int len = 0;  
        for (i = 0; i < biglist.size() - 1; i++) {  
            len += biglist.get(i);  
            map.put (len, map.getOrDefault(len, 0) + 1);  
            ans = Math.max (ans, map.get(len));  
        }  
    }  
    return wall.size() - ans;  
}
```

T.C: $O(\text{no. of elements})$
S.C: $O(\text{no. of elements})$

```

HashMap< Integer, Integer> map = new HashMap<>();
int ans = 0;
for (List< Integer> bigList : walls) {
    int len = 0;
    for (i = 0; i < bigList.size() - 1; i++) {
        len += bigList.get(i);
        map.put (len, map.getOrDefault(len, 0) + 1);
        ans = Math.max (ans, map.get (len));
    }
}

```

1-123 2-1
 3-123
 5-12
 4-1234



4
 ans = 0 + 1 + 2 = 3

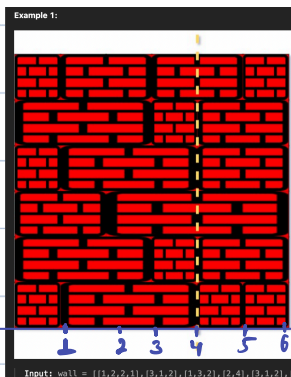
→ Biglist: $\begin{array}{r} 0 \ 1 \ 2 \ 3 \\ \hline 1 \ 2 \ 2 \ 1 \end{array}$
 $len = 0 + 1 + 2 = 3 + 2 = 5$

→ Biglist: $\begin{array}{r} 0 \ 1 \ 2 \\ \hline 3 \ 1 \ 2 \end{array}$
 $len = 0 + 2 + 1 = 3$

→ Biglist: $\begin{array}{r} 0 \ 1 \ 2 \\ \hline 1 \ 3 \ 2 \end{array}$
 $len = 0 + 1 + 2 + 3 = 6$

→ Biglist: $\begin{array}{r} 2 \ 4 \end{array}$
 $len = 0 + 2 = 2$

→ Biglist: $\begin{array}{r} 3 \ 1 \ 2 \end{array}$
 $len = 0 + 3 + 1 = 4$



walls

0	1	2	3	4	5
1	2	1	2	1	1
2	1	3	4	1	2
2	2	2			
1					

→ Biglist: $\begin{array}{r} 1 \ 3 \ 1 \ 1 \end{array}$
 $len = 0 + 1 + 3 + 4 + 1 = 9$



Q) Longest consecutive sequence

↳ Given array ele, find the length of longest sequence which can be rearranged in a strictly increasing by 1.

Ex: arr[]: ^{0 1 2 3 4 5 6 7} {-1 8 5 3 10 2 4 9}

arr[]: ^{0 1 2 3 4 5 6 7 8 9} {3 8 2 1 9 6 5 6 7 2}



AlgoPrep